

DEEP LEARNING

CNN & Transformers

Chrys NIONGOLO

Introduction technique · Master 2 Intelligence artificielle

MASTER 2

35 h

5 JOURS



- 01 Réseaux profonds & rétropropagation
- 02 Optimisation & convolutions
- 03 CNN & transfert d'apprentissage
- 04 Attention & Transformers
- 05 Modèles causaux & mise en pratique

APERÇU DU COURS

Un calcul, une implémentation et une expérience pour comprendre chaque architecture.

Cours, exercices et cinq TP progressifs sur CPU.



5 journées de 7 heures



5 TP progressifs

SOMMAIRE

MASTER 2

35 h

5 JOURS



OBJECTIFS ET PREUVES DE MAÎTRISE

Objectif	Preuve attendue
Rétropropagation	Dérivation, dimensions et contrôle numérique
CNN pour la vision	Architecture, entraînement et analyse des erreurs
Transfert	Comparaison source / cible et stratégies de gel
Transformer	Attention, masque causal et boucle autoregressive
Démarche expérimentale	Validation séparée, ablations et limites explicites

SPE
DATA/IA

PRÉREQUIS ET OUTILS

Algèbre linéaire : produit matriciel, transposée, normes.

Analyse : dérivées partielles et règle de la chaîne.

Probabilités : vraisemblance, logarithme, espérance.

Python, NumPy et bases de l'évaluation supervisée.



CONVENTIONS DE NOTATION

$$x \in \mathbb{R}^d, \quad W_\ell \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$$
$$X \in \mathbb{R}^{B \times d}, \quad H_\ell = \phi(XW_\ell^\top + \mathbf{1}b_\ell^\top)$$



B : batch · n : longueur · d : dimension · K : classes

Un produit matriciel doit toujours être vérifié par ses dimensions.



01 / RÉSEAUX PROFONDS

Calcul direct, pertes et rétropropagation



JOUR 1 · DÉROULÉ DES 7 HEURES

Séquence	Travail attendu	Minutes
Cadrage	Pré requis, notation et formulation du problème	45
Modélisation	MLP, activations et fonctions de perte	90
Dérivations	Règle de la chaîne et gradients vectorisés	75
TP 01	MLP NumPy, gradient numérique et généralisation	150
Restitution	Autograd, exercices et synthèse	60

Total : 420 minutes · 7 heures de formation effective, hors pauses



APPRENDRE DES REPRÉSENTATIONS

$$f_{\theta} = f_L \circ f_{L-1} \circ \dots \circ f_1$$



Un réseau compose des transformations paramétrées.

Les couches internes construisent des représentations utiles à la tâche.

La profondeur apporte une structure de composition ; elle ne garantit pas la performance.



LE NEURONE DIFFÉRENTIABLE

$$z = w^{\top}x + b, \quad a = \phi(z)$$



Combinaison affine : poids w et biais b .

Activation : une transformation non linéaire choisie.

Les paramètres sont appris ; l'architecture est un choix de modélisation.



POURQUOI LA NON-LINÉARITÉ ?

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$



Empiler des couches affines reste une transformation affine.

Les activations rendent possibles des frontières de décision non linéaires.

La profondeur seule ne suffit pas si toutes les activations sont l'identité.

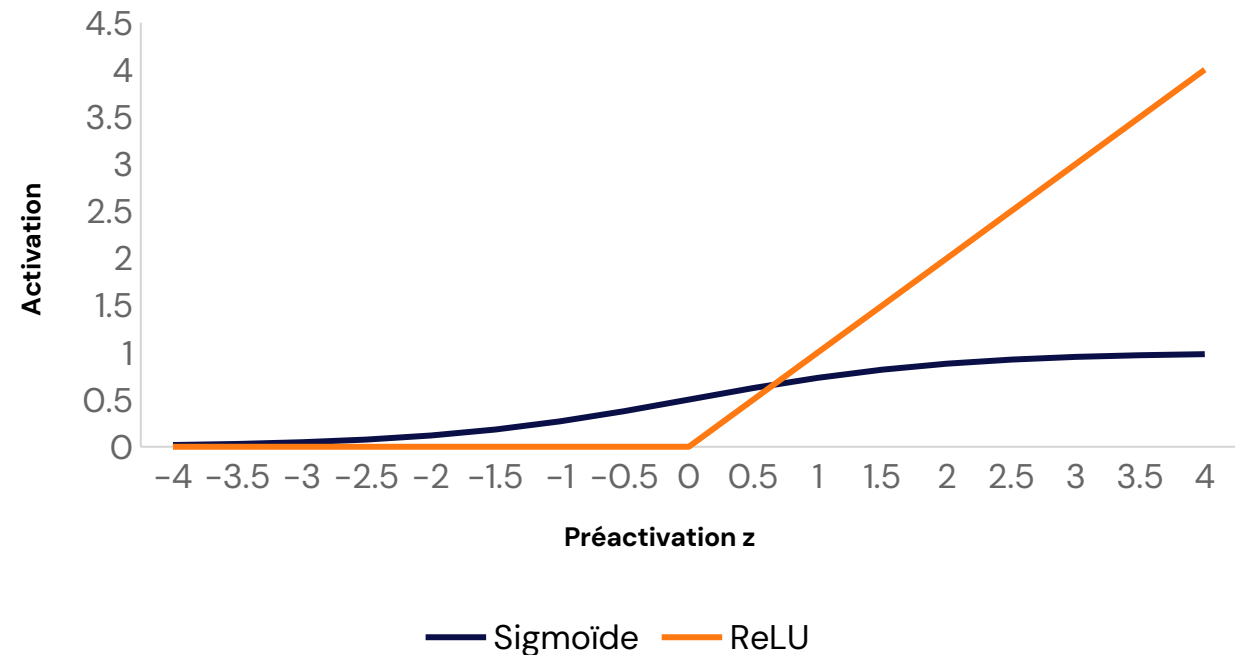


ACTIVATIONS : VALEURS ET SATURATION

La sigmoïde borne la sortie entre 0 et 1.

ReLU conserve les valeurs positives et annule les négatives.

Le choix influence le signal transmis et ses gradients.



Courbes calculées à partir des définitions.



DÉRIVÉES DES ACTIVATIONS

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \quad \phi'_{\text{ReLU}}(z) = \mathbf{1}_{z>0}$$
$$\text{GELU}(z) = z\Phi(z)$$



La règle de la chaîne multiplie ces dérivées locales.

Pour ReLU en zéro, les logiciels adoptent une convention.

GELU constitue une activation lisse courante dans les Transformers.



UNE COUCHE DENSE : FORMES ET CALCUL

$$a_\ell = \phi_\ell(W_\ell a_{\ell-1} + b_\ell), \quad P_\ell = d_\ell(d_{\ell-1} + 1)$$



Chaque neurone lit toutes les composantes de la couche précédente.

Un biais est partagé entre les exemples, pas entre les neurones.

Le calcul batch utilise des opérations matricielles.



MLP : ARCHITECTURE ET PARAMÈTRES

Opération	Forme pour un batch B	Paramètres
Entrée	$B \times 2$	0
Dense + ReLU	$B \times 16$	$2 \times 16 + 16 = 48$
Dense : logits	$B \times 2$	$16 \times 2 + 2 = 34$
Softmax pour lecture	$B \times 2$	0
Total	2 classes	82

SPE
DATA/IA

PERTE DE RÉGRESSION

$$\ell(\hat{y}, y) = \frac{1}{2} \|\hat{y} - y\|_2^2, \quad \nabla_{\hat{y}} \ell = \hat{y} - y$$



Une perte compare une prédiction à sa cible.

Le facteur 1/2 simplifie la dérivée, sans changer le minimiseur.

Une erreur quadratique pénalise fortement les grands résidus.



CLASSIFICATION : SOFTMAX ET ENTROPIE CROISÉE

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad \ell(z, y) = - \sum_{k=1}^K y_k \log p_k$$



Les logits sont des scores réels non normalisés.

Softmax produit une distribution sur K classes.

La perte favorise la probabilité de la classe correcte.



STABILITÉ NUMÉRIQUE DES LOGITS

$$\ell(z, c) = -z_c + m + \log \sum_j e^{z_j - m}, \quad m = \max_j z_j$$



Soustraire le maximum ne change pas softmax.

La forme log-sum-exp évite de calculer de grandes exponentielles.

CrossEntropyLoss reçoit directement les logits.



RISQUE EMPIRIQUE ET GÉNÉRALISATION

$$\hat{R}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i), \quad \hat{\theta} \approx \arg \min_{\theta} \hat{R}(\theta)$$



L'objectif d'entraînement agrège les pertes sur les exemples observés.

La validation sert au choix des modèles et hyperparamètres.

Le test mesure le modèle choisi, sans piloter ses réglages.



DESCENTE DE GRADIENT

$$\theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \hat{R}(\theta_t)$$



Le gradient indique la variation locale la plus forte.

Le taux η règle la longueur du pas.

Rétropropagation et optimisation sont deux opérations distinctes.



RÈGLE DE LA CHAÎNE

$$z = g(x), \quad u = h(z), \quad \frac{\partial \ell}{\partial x} = \frac{\partial \ell}{\partial u} \frac{\partial u}{\partial z} \frac{\partial z}{\partial x}$$



Un calcul composé se dérive opération par opération.

Le passage arrière réutilise les valeurs du passage avant.

Les contributions issues de plusieurs chemins s'additionnent.



EXEMPLE SCALAIRE : UN PAS COMPLET

$$\ell = \frac{1}{2}(a - y)^2, \quad a = \sigma(wx + b)$$
$$\frac{\partial \ell}{\partial w} = (a - y)a(1 - a)x \approx -0.1058$$



Entrée $x=2$, cible $y=1$, poids $w=0,5$ et biais $b=0$.

Sortie sigmoïde $a \approx 0,7311$; perte quadratique $\approx 0,0362$.

Avec $\eta=0,1$, le poids devient $\approx 0,5106$.



GRADIENT DE SOFTMAX + ENTROPIE CROISÉE

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j), \quad \frac{\partial \ell}{\partial z} = p - y$$



La dérivée de softmax couple toutes les classes.

La combinaison avec l'entropie croisée se simplifie.

Le résultat porte sur les logits, pas sur les probabilités.



RÉTROPROPAGATION D'UNE COUCHE DENSE

$$\delta_\ell = \frac{\partial \ell}{\partial z_\ell}, \quad \nabla_{W_\ell} \ell = \delta_\ell a_{\ell-1}^\top$$
$$\nabla_{b_\ell} \ell = \delta_\ell, \quad \frac{\partial \ell}{\partial a_{\ell-1}} = W_\ell^\top \delta_\ell$$

SPE
DATA/IA

δ désigne le gradient de la perte par rapport à la préactivation.

Le gradient de W est un produit extérieur.

Le gradient transmis à l'entrée utilise la transposée de W .



PROPAGER LE SIGNAL DANS LES COUCHES CACHÉES

$$\delta_\ell = \left(W_{\ell+1}^\top \delta_{\ell+1} \right) \odot \phi'_\ell(z_\ell)$$



Revenir depuis la couche suivante par $W^\top$.

Appliquer ensuite la dérivée de l'activation locale.

Conserver les préactivations ou les informations permettant de retrouver leur dérivée.



GRADIENTS VECTORISÉS SUR UN BATCH

$$Z = HW^{\top} + \mathbf{1}b^{\top}, \quad D = \frac{\partial \mathcal{L}}{\partial Z}$$

$$\nabla_W \mathcal{L} = D^{\top} H, \quad \nabla_b \mathcal{L} = \sum_{i=1}^B D_{i,:}, \quad \nabla_H \mathcal{L} = DW$$

SPE
DATA/IA

Les exemples sont des lignes et les poids restent sortie × entrée.

Le biais est agrégé sur l'axe batch.

La convention de moyenne doit être cohérente dans tout le code.



VÉRIFIER UN GRADIENT NUMÉRIQUEMENT

$$g_j^{\text{num}} = \frac{\mathcal{L}(\theta + \varepsilon e_j) - \mathcal{L}(\theta - \varepsilon e_j)}{2\varepsilon}$$
$$r = \frac{\|g^{\text{num}} - g\|_2}{\|g^{\text{num}}\|_2 + \|g\|_2 + 10^{-12}}$$



Comparer la dérivée analytique à une différence finie centrée.

Travailler en float64 et sur un très petit problème.

Éviter les points où une activation n'est pas différentiable.



AUTOGRAD : CE QUI EST AUTOMATISÉ

```
optimizer.zero_grad(set_to_none=True)
logits = model(x)
loss = criterion(logits, y)
loss.backward()
optimizer.step()
```



Le graphe enregistre les opérations sur les tenseurs.

`backward()` accumule les gradients des paramètres feuilles.

L'optimiseur met à jour les paramètres après le calcul du gradient.



BOUCLE D'ENTRAÎNEMENT : INVARIANTS

Une époque parcourt le jeu d'entraînement.

Chaque batch déclenche un passage avant, arrière, puis une mise à jour.

La validation s'exécute sans mise à jour des paramètres.



TP 01 · UN MLP EN NUMPY



CONTEXTE

TP 01 · DONNÉES

Deux lunes en 2D. Architecture $2 \rightarrow 16 \rightarrow 2$.



EXERCICE

TRAVAIL DEMANDÉ

Relier forward, backward et différences finies. Varier largeur et taux d'apprentissage.



BILAN

LIVRABLES

Courbes train/validation, gradients vérifiés et compte rendu des trois expériences.

SPE
DATA/IA

RÉSULTATS ATTENDUS



150 min

TEMPS DE L'EXERCICE



2

ÉTUDIANTS PAR BINÔME



3

PREUVES ATTENDUES



DIAGNOSTIQUER AVANT D'AJOUTER DES COUCHES

Symptôme	Vérification prioritaire
Perte constante	Gradients, taux d'apprentissage, paramètres entraînaibles
NaN ou inf	Logits, normalisation, division et pas trop grand
Train bon, validation faible	Split, surapprentissage, décalage de distribution
Score presque parfait dès le début	Fuite de cible ou duplication train/test
Gradients NumPy $\neq$ numériques	Axes, transpose, facteur de moyenne et ReLU

SPE
DATA/IA

EXERCICE · RECONSTRUIRE LE GRADIENT

Réseau : $3 \rightarrow 4 \rightarrow 2$, avec ReLU et perte cross-entropy.

Donner la forme de chaque poids, activation et gradient.

Calculer le nombre de paramètres et justifier chaque transposée.



JOUR 1 · À RETENIR

Une architecture, une perte et un optimiseur jouent des rôles distincts.

La rétropropagation réutilise les dérivées locales et additionne les chemins.

Les dimensions et le contrôle numérique rendent les gradients vérifiables.

Une perte d'entraînement faible ne démontre pas la généralisation.



02/ OPTIMISATION ET CONVOLUTIONS

Stabiliser l'apprentissage et exploiter la structure spatiale



JOUR 2 · DÉROULÉ DES 7 HEURES

Séquence	Travail attendu	Minutes
Optimisation	Mini-batches, Adam, initialisation et régularisation	90
CNN	Convolution, formes, paramètres et pooling	105
Exercices	Calcul à la main et champ réceptif	45
TP 02	Petit CNN sur digits et expériences contrôlées	150
Synthèse	Diagnostic et restitution	30

Total : 420 minutes · 7 heures de formation effective, hors pauses



SGD ET MINI-BATCHES

$$g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} \ell_i(\theta_t), \quad \theta_{t+1} = \theta_t - \eta_t g_t$$



Le gradient du batch approche celui du jeu complet.

B change le bruit, le coût par étape et la mémoire.

Comparer des expériences à budget explicite : étapes, époques et temps.



MOMENTUM : MÉMORISER UNE DIRECTION

$$v_t = \beta v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t$$



Une moyenne temporelle amortit les changements rapides de gradient.

Le coefficient β contrôle la mémoire des gradients passés.

Les conventions de normalisation diffèrent selon les formulations.



ADAM : PREMIER ET SECOND MOMENTS

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t / (1 - \beta_1^t)}{\sqrt{v_t / (1 - \beta_2^t)} + \varepsilon}$$

SPE
DATA/IA

Adam suit une moyenne du gradient et de son carré.

Les corrections de biais compensent l'initialisation à zéro.

La division est élément par élément.



INITIALISATION ET PROPAGATION DES VARIANCES

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{in}}}\right) \quad (\text{ReLU}), \quad \text{Var}(W_{ij}) \approx \frac{2}{d_{\text{in}} + d_{\text{out}}} \quad (\text{Xavier})$$



Des poids tous identiques préservent des symétries indésirables.

L'échelle doit tenir compte du nombre d'entrées.

Pour ReLU, l'initialisation de He vise une variance adaptée.



GRADIENTS QUI DISPARAISSENT OU EXPLOSENT

$$\frac{\partial \ell}{\partial a_0} = J_1^\top J_2^\top \cdots J_L^\top \frac{\partial \ell}{\partial a_L}$$



Le gradient profond contient un produit de Jacobiennes.

L'échelle des poids et les dérivées d'activation agissent ensemble.

Initialisation, résidus, normalisation et clipping répondent à des mécanismes différents.



PÉNALISATION L2 ET WEIGHT DECAY

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \|\theta\|_2^2, \quad \nabla \mathcal{L}_{\text{reg}} = \nabla \mathcal{L} + \lambda \theta$$



L2 ajoute une pénalité à la fonction objectif.

Avec SGD simple, une convention cohérente donne une décroissance des poids.

Avec un optimiseur adaptatif, AdamW découple cette décroissance.



DROPOUT : TRAIN ET ÉVALUATION

$$m_j \sim \text{Bernoulli}(1 - p), \quad \tilde{a}_j = \frac{m_j}{1 - p} a_j, \quad \mathbb{E}[\tilde{a}_j] = a_j$$



Au train, un masque aléatoire retire des activations.

La version inversée conserve leur espérance en les redimensionnant.

À l'évaluation, le masque est désactivé.



NORMALISER LES ACTIVATIONS

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}}, \quad y = \gamma \hat{x} + \beta$$



Normaliser ne signifie pas retirer toute l'information d'échelle.

Des paramètres γ et β réintroduisent une transformation affine apprise.

Les axes de calcul des statistiques dépendent du type de normalisation.

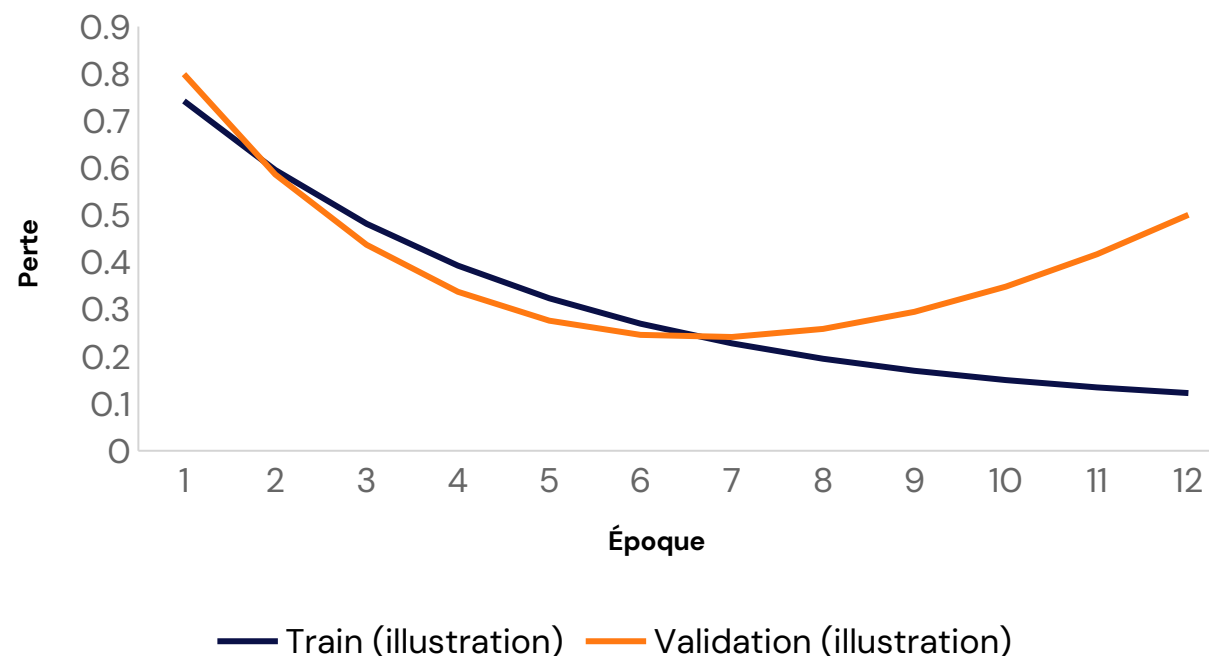


EARLY STOPPING ET COURBES D'APPRENTISSAGE

La validation choisit un checkpoint selon un critère annoncé.

La patience tolère des fluctuations temporaires.

Le test est réservé à l'évaluation du checkpoint retenu.



Courbes illustratives ; elles ne représentent pas un entraînement mesuré.



POURQUOI UN CNN POUR UNE IMAGE ?

Les voisinages locaux ont une structure spatiale.

Un même détecteur peut être utile en plusieurs positions.

La convolution impose localité et partage des paramètres.



CONVOLUTION 2D : L'OPÉRATION LOCALE

$$Y_{o,i,j} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} W_{o,c,u,v} X_{c,i+u,j+v}$$



Chaque sortie combine un voisinage spatial et tous les canaux d'entrée.

Le noyau est partagé entre les positions.

Les bibliothèques emploient généralement une corrélation croisée.



CONVOLUTION : EXEMPLE À LA MAIN

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$
$$Y = \begin{bmatrix} 0 & -1 \\ -1 & 1 \end{bmatrix}$$



Entrée 3×3, noyau 2×2, stride 1 et aucun padding.

Le noyau n'est pas retourné dans cet exemple.

La sortie est 2×2.



STRIDE, PADDING ET DILATION

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - d(k - 1) - 1}{s} + 1 \right\rfloor$$



Stride s : espacement des positions de sortie.

Padding p : extension du bord de l'entrée.

Dilation d : espacement des coefficients du noyau.



CANAUX ET TENSEURS D'UN CNN

Objet	Forme	Rôle
Entrée	$B \times C_{in} \times H \times W$	Images du batch
Poids	$C_{out} \times C_{in} \times k_h \times k_w$	Détecteurs partagés
Biais	C_{out}	Décalage par canal de sortie
Sortie	$B \times C_{out} \times H_{out} \times W_{out}$	Cartes de caractéristiques

SPE
DATA/IA

TAILLES ET NOMBRE DE PARAMÈTRES

$$P = C_{\text{out}}(C_{\text{in}}k_hk_w + 1)$$

$$\text{MACs} \approx H_{\text{out}}W_{\text{out}}C_{\text{out}}C_{\text{in}}k_hk_w$$



Convolution dense en canaux, sans groupes.

Le nombre de positions ne multiplie pas les paramètres.

Le coût de calcul, lui, dépend de la taille spatiale.



POOLING : RÉDUIRE LA RÉOLUTION

$$\text{maxpool} \begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix} = 4, \quad \text{avgpool} \begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix} = 2.5$$



Max pooling conserve la valeur maximale d'une fenêtre.

Average pooling calcule sa moyenne.

Le pooling usuel ne possède pas de paramètres appris.



CHAMP RÉCEPTIF : CALCUL RÉCURSIF

$$j_\ell = j_{\ell-1} s_\ell, \quad r_\ell = r_{\ell-1} + (k_\ell - 1) d_\ell j_{\ell-1}$$
$$r_0 = j_0 = 1$$



r : largeur du champ réceptif théorique.

j : pas, dans l'image d'origine, entre deux sorties voisines.

Le champ théorique et l'influence effective apprise ne sont pas identiques.



ÉQUIVARIANCE ET INVARIANCE

$$f(Tx) = Tf(x) \quad (\text{equivariance}), \quad g(Tx) = g(x) \quad (\text{invariance})$$



Équivariance : déplacer l'entrée déplace la représentation.

Invariance : déplacer l'entrée ne change pas la sortie.

Stride, bords et lecture finale limitent les propriétés exactes.



TP 02 · CONSTRUIRE UN PETIT CNN



CONTEXTE

TP 02 · DONNÉES

Chiffres digits : images 8×8, dix classes.
CPU.



EXERCICE

TRAVAIL DEMANDÉ

Calculer les formes et paramètres, entraîner
puis modifier un seul choix à la fois.



BILAN

LIVRABLES

Courbes, matrice de confusion et contrôle
du nombre de paramètres.

SPE
DATA/IA

RÉSULTATS ATTENDUS



150 min

TEMPS DE L'EXERCICE



2

ÉTUDIANTS PAR BINÔME



3

PREUVES ATTENDUES



EXERCICE · AUDITER UNE CONVOLUTION

Entrée : $B \times 3 \times 32 \times 32$; sortie : 16 canaux.

Noyau 3×3 , padding 1, stride 2, dilation 1.

Calculer forme, paramètres et multiplications-accumulations par image.



JOUR 2 · À RETENIR

Un gradient correct peut être mal exploité par l'optimiseur.

Une convolution partage des paramètres locaux entre les positions.

Dimensions, paramètres, MACs et mémoire sont des quantités différentes.

Pooling et stride réduisent la résolution et perdent de l'information.



03/ CNN ET TRANSFERT

Architecture, données, évaluation et adaptation



JOUR 3 · DÉROULÉ DES 7 HEURES

Séquence	Travail attendu	Minutes
Architectures	CNN complet, BatchNorm et résidus	60
Données	Augmentation, séparation et métriques	75
Transfert	Gel, adaptation et expériences comparables	75
TP 03	Source digits 0–4 vers cible digits 5–9	180
Restitution	Erreurs, résultats et limites	30

Total : 420 minutes · 7 heures de formation effective, hors pauses



LE CNN DU TP : PARCOURS D'UN BATCH

Étape	Forme hors batch	Paramètres
Conv 3×3 + ReLU	$8 \times 8 \times 8$	80
MaxPool 2×2	$8 \times 4 \times 4$	0
Conv 3×3 + ReLU	$16 \times 4 \times 4$	1 168
MaxPool + flatten	64	0
Linear 64 → 10	10 logits	650
Total		1 898

SPE
DATA/IA



CONVOLUTION 1×1 ET AGRÉGATION GLOBALE

$$Y_{o,i,j} = b_o + \sum_c W_{o,c} X_{c,i,j}, \quad g_c = \frac{1}{HW} \sum_{i,j} X_{c,i,j}$$



Une convolution 1×1 mélange les canaux à chaque position.

Elle peut changer la largeur sans regarder les pixels voisins.

Global average pooling transforme chaque carte en une valeur.



BATCHNORM DANS UN CNN

$$\mu_c = \frac{1}{BHW} \sum_{b,i,j} X_{b,c,i,j}, \quad \hat{X}_{b,c,i,j} = \frac{X_{b,c,i,j} - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}}$$



Au train, les statistiques sont calculées par canal sur B×H×W.

Des statistiques mobiles sont utilisées à l'évaluation standard.

Un petit batch ou un changement de domaine peut perturber ce mécanisme.



CONNEXIONS RÉSIDUELLES

$$y = x + F(x), \quad \frac{\partial y}{\partial x} = I + \frac{\partial F}{\partial x}$$



Un bloc apprend une correction $F(x)$ ajoutée à l'entrée.

L'addition exige des formes compatibles.

Le chemin identité contribue directement à la dérivée.



AUGMENTER SANS CHANGER LA CIBLE

$$\mathcal{L}_{\text{aug}} = \mathbb{E}_{(x,y)} \mathbb{E}_{T \sim \mathcal{A}}[\ell(f_{\theta}(Tx), y)]$$



Une augmentation encode une transformation supposée pertinente.

Elle s'applique au train ; l'évaluation garde un protocole défini.

La validité dépend de la tâche, pas de la popularité de la transformation.



LE JEU DE DONNÉES FAIT PARTIE DU MODÈLE

Vérifier classes, doublons, résolution, normalisation et ambiguïtés.

Définir l'unité indépendante : image, personne, appareil ou document.

Documenter les limites du jeu de démonstration.



TRAIN, VALIDATION ET TEST : QUI DÉCIDE ?

Partition	Utilisation	À éviter
Train	Poids et statistiques apprises	Information issue du test
Validation	Modèle, hyperparamètres et checkpoint	Confondre sélection et estimation finale
Test	Évaluer la procédure déjà choisie	Choisir les réglages selon son score
Nouvelle distribution	Vérifier le transfert externe	La remplacer par le seul test interne

SPE
DATA/IA



MÉTRIQUES ET MATRICE DE CONFUSION

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{rappel} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}$$



L'accuracy agrège toutes les décisions correctes.

Le rappel et la précision se lisent classe par classe.

Macro-F1 donne le même poids aux classes, sans effacer toutes les limites.



TRANSFERT : RÉUTILISER UNE REPRÉSENTATION

$$f(x) = h_{\psi}(g_{\phi}(x)), \quad \phi \leftarrow \phi_{\text{source}}, \quad \psi \leftarrow \psi_{\text{nouvelle}}$$



Un backbone appris sur une tâche source initialise la tâche cible.

La tête finale est adaptée au nouvel espace de sorties.

Un transfert positif dépend de la relation entre source et cible.



TROIS STRATÉGIES DE TRANSFERT

Stratégie	Backbone	Tête	Usage / limite
Depuis zéro	Aléatoire, entraîné	Entraînée	Référence de comparaison
Features gelées	Source, figé	Entraînée	Peu de labels, capacité d'adaptation limitée
Fine-tuning	Source, adapté	Entraînée	Plus flexible, risque de surapprentissage

SPE
DATA/IA



FINE-TUNING : UNE PROCÉDURE EXPLICITE

Initialiser le backbone source et une nouvelle tête.

Entraîner la tête avec le backbone gelé.

Dégeler les couches choisies et réduire leur taux d'apprentissage.

Sélectionner le checkpoint sur la validation cible.



GEL DES POIDS ET MODE ÉVALUATION

```
for p in backbone.parameters():  
    p.requires_grad_(False)  
backbone.eval()  
optimizer = torch.optim.Adam(head.parameters(), lr=1e-3)
```



`requires_grad` contrôle le calcul des gradients des paramètres.

`eval()` contrôle Dropout et les statistiques de normalisation.

Un backbone gelé peut néanmoins changer de comportement s'il reste en mode train.



UTILISER UN MODÈLE PRÉENTRAÎNÉ PUBLIC

```
from torchvision.models import resnet18, ResNet18_Weights
weights = ResNet18_Weights.IMAGENET1K_V1
model = resnet18(weights=weights)
preprocess = weights.transforms()
model.fc = torch.nn.Linear(model.fc.in_features, K)
```



Choisir explicitement la version des poids.

Appliquer le prétraitement associé aux poids.

Adapter la tête et les classes de la tâche cible.



COMPARER DES EXPÉRIENCES

À conserver	Pourquoi
Split et graine	Comparer sur les mêmes observations
Architecture et paramètres	Identifier la capacité entraînée
Optimiseur, pas et batch	Reproduire le budget d'apprentissage
Critère de checkpoint	Comprendre la sélection
Score, effectifs et temps	Interpréter performance et coût

SPE
DATA/IA



ANALYSE D'ERREURS : REGARDER LES EXEMPLES

Regrouper les erreurs par classe et par type de difficulté.

Examiner les cas confiants mais faux.

Vérifier annotation, ambiguïté et décalage de distribution.



TP 03 · MESURER LE TRANSFERT



CONTEXTE

TP 03 · DONNÉES

Source : digits 0–4. Cible : digits 5–9, avec peu d'exemples étiquetés.



EXERCICE

TRAVAIL DEMANDÉ

Comparer entraînement depuis zéro, features gelées et fine-tuning.



BILAN

LIVRABLES

Même split cible, budget explicite, validation séparée et bilan des limites.

SPE
DATA/IA

RÉSULTATS ATTENDUS



180 min

TEMPS DE L'EXERCICE



2

ÉTUDIANTS PAR BINÔME



3

PREUVES ATTENDUES



JOUR 3 · À RETENIR

Les représentations peuvent être réutilisées, avec des limites de domaine.

Geler les poids et figer le comportement des couches sont deux décisions.

La comparaison exige une baseline, un budget et une règle de sélection.

L'analyse des erreurs complète les métriques agrégées.



04/ ATTENTION ET TRANSFORMERS

Calcul matriciel, positions, masques et architecture



JOUR 4 · DÉROULÉ DES 7 HEURES

Séquence	Travail attendu	Minutes
Séquences	Représentation, récurrence et positions	60
Attention	Q, K, V, softmax et masques	90
Architecture	Têtes, normalisation et blocs Transformers	90
TP 04	Calcul d'attention et tests de causalité	150
Synthèse	Dimensions, coût et questions de contrôle	30

Total : 420 minutes · 7 heures de formation effective, hors pauses



UNE SÉQUENCE COMME TENSEUR

$$X \in \mathbb{R}^{B \times n \times d}$$



Un token est une unité de représentation, pas nécessairement un mot.

Une séquence possède une longueur n et une largeur de représentation d .

Le padding uniformise les tailles dans un batch, mais doit être masqué.



RÉCURRENCE ET DÉPENDANCES LONGUES

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b)$$



Un RNN met à jour un état en parcourant la séquence.

La dépendance temporelle impose un calcul séquentiel des états.

L'attention permet des interactions directes entre positions.



EMBEDDINGS : DES IDENTIFIANTS AUX VECTEURS

$$E \in \mathbb{R}^{V \times d}, \quad x_t = E[\text{id}(t), :]$$



La table d'embedding contient un vecteur appris par token du vocabulaire.

Une recherche de ligne remplace la multiplication explicite par un one-hot.

L'embedding est entraîné avec le reste du modèle, sauf gel explicite.



INFORMATION DE POSITION

$$X_0 = E[\text{tokens}] + P, \quad \text{SA}(\Pi X) = \Pi \text{SA}(X)$$



L'auto-attention seule ne code pas l'ordre absolu des tokens.

On peut ajouter des positions apprises ou déterministes.

Sans position, une permutation de l'entrée permute aussi la sortie de l'auto-attention non masquée.



ENCODAGE SINUSOÏDAL DES POSITIONS

$$PE_{t,2i} = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad PE_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right)$$



Chaque paire de dimensions utilise une fréquence.

Les positions ont la même largeur d que les embeddings.

Cette construction ne garantit pas, à elle seule, la généralisation à de longues séquences.



L'ATTENTION COMME SOMME PONDÉRÉE

$$o_i = \sum_{j=1}^{n_k} \alpha_{ij} v_j, \quad \sum_j \alpha_{ij} = 1, \quad \alpha_{ij} \geq 0$$



Une requête attribue un score à chaque clé.

Softmax normalise les scores sur les positions consultées.

La sortie est une combinaison des valeurs.



Q, K, V : PROJECTIONS APPRISES

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$
$$W_Q, W_K \in \mathbb{R}^{d \times d_k}, \quad W_V \in \mathbb{R}^{d \times d_v}$$



Q et K définissent l'espace de comparaison.

V définit le contenu qui sera agrégé.

En auto-attention, les trois projections partent de la même séquence.



SCORES D'ATTENTION : TOUTES LES PAIRES

$$S = \frac{QK^\top}{\sqrt{d_k}}, \quad S_{ij} = \frac{q_i^\top k_j}{\sqrt{d_k}}$$



Une ligne correspond à une requête.

Une colonne correspond à une clé.

Le produit $QK^\top$ produit une matrice $n_q \times n_k$.



POURQUOI DIVISER PAR $\sqrt{d_k}$?

$$\text{Var} \left(\sum_{r=1}^{d_k} q_r k_r \right) \approx d_k \quad \implies \quad \text{Var} \left(\frac{q^\top k}{\sqrt{d_k}} \right) \approx 1$$



Sous des hypothèses simplifiées, la variance d'un produit scalaire croît avec d_k .

Des scores trop dispersés peuvent saturer softmax.

La normalisation stabilise l'échelle, sans prouver à elle seule la stabilité du réseau.



SOFTMAX PUIS AGRÉGATION DES VALEURS

$$A = \text{softmax}_{\text{lignes}}(S), \quad O = AV \in \mathbb{R}^{n_q \times d_v}$$



Softmax est appliqué indépendamment à chaque ligne des scores.

Les poids A ont la même forme que S.

La sortie retrouve n_q positions, avec d_v caractéristiques.



ATTENTION : EXEMPLE NUMÉRIQUE COMPLET

$$Q = K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$$
$$A \approx \begin{bmatrix} 0.6698 & 0.3302 \\ 0.3302 & 0.6698 \end{bmatrix}, \quad O \approx \begin{bmatrix} 0.6698 & 0.6605 \\ 0.3302 & 1.3395 \end{bmatrix}$$

SPE
DATA/IA

Deux requêtes, deux clés, $d_k=2$ et deux composantes par valeur.

Les poids sont arrondis à quatre décimales.

La somme de chaque ligne vaut 1.



MASQUE CAUSAL ET MASQUE DE PADDING

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}, \quad A = \text{softmax}(S + M)$$



Le masque causal interdit de consulter les positions futures.

Le masque de padding exclut les clés ajoutées artificiellement.

Les scores interdits reçoivent $-\infty$ avant softmax.



MULTI-HEAD ATTENTION

$$\text{head}_r = \text{Attn}(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)})$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$



Chaque tête dispose de ses projections Q, K et V.

Les résultats sont concaténés puis projetés.

Plusieurs têtes ne signifient pas plusieurs interprétations humaines garanties.



DÉPLIAGE DES AXES MULTI-TÊTES

Étape	Forme standard	Exemple $B=2, n=4, d=8, h=2$
Projection Q/K/V	$B \times n \times d$	$2 \times 4 \times 8$
Séparation des têtes	$B \times h \times n \times d_h$	$2 \times 2 \times 4 \times 4$
Scores	$B \times h \times n \times n$	$2 \times 2 \times 4 \times 4$
Mélange des valeurs	$B \times h \times n \times d_h$	$2 \times 2 \times 4 \times 4$
Concaténation	$B \times n \times d$	$2 \times 4 \times 8$

SPE
DATA/IA



LAYERNORM : NORMALISER CHAQUE TOKEN

$$\mu_i = \frac{1}{d} \sum_{r=1}^d X_{ir}, \quad \text{LN}(X_i) = \gamma \odot \frac{X_i - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}} + \beta$$



Les statistiques portent sur les d caractéristiques d'un token.

Aucune moyenne sur le batch n'est nécessaire dans cette configuration.

Le même calcul s'applique au train et à l'inférence.



LE MLP POSITION PAR POSITION

$$\text{FFN}(x) = W_2 \phi(W_1 x + b_1) + b_2, \quad d \rightarrow d_{\text{ff}} \rightarrow d$$



Le FFN transforme les caractéristiques de chaque token séparément.

Il élargit souvent la dimension avant de la réduire.

Les mêmes poids sont partagés entre toutes les positions.



UN BLOC TRANSFORMER PRE-LN

$$U = X + \text{MHA}(\text{LN}_1(X))$$

$$Y = U + \text{FFN}(\text{LN}_2(U))$$



Chaque sous-couche est précédée d'une LayerNorm.

Les additions résiduelles conservent la forme $B \times n \times d$.

Une normalisation finale peut compléter la pile.



ENCODEUR : CONTEXTUALISER UNE SÉQUENCE

$$X_L = \text{Block}_L \circ \dots \circ \text{Block}_1(X_0)$$



L'attention peut consulter toutes les positions non masquées.

La pile produit une représentation contextualisée par token.

Une tête adapte cette représentation à la tâche.



CROSS-ATTENTION : RELIER DEUX SÉQUENCES

$$Q = X_{\text{cible}} W_Q, \quad K = H_{\text{source}} W_K, \quad V = H_{\text{source}} W_V$$



Les requêtes proviennent de la séquence cible.

Les clés et valeurs proviennent de la mémoire source.

Les longueurs cible et source peuvent différer.



TROIS FAMILLES D'ARCHITECTURES

Famille	Accès aux positions	Exemple de tâche
Encodeur	Toutes les positions disponibles	Classification / étiquetage
Décodeur seul	Passé et position courante	Prédiction du prochain token
Encodeur-décodeur	Source complète + cible causale	Transformation d'une séquence en une autre

SPE
DATA/IA



COÛT DE L'ATTENTION DENSE

$$T_{\text{attn}} = O(n^2 d), \quad T_{\text{proj+FFN}} = O(nd^2)$$

$$M_{\text{scores}} = O(Bhn^2) \quad \text{si les scores sont matérialisés}$$



Les interactions croissent quadratiquement avec n .

Les projections et le FFN ajoutent un coût dépendant de d^2 .

Les implémentations optimisées peuvent réduire la mémoire matérialisée.



ATTENTION MANUELLE : LE CŒUR DU CODE

```
scores = q @ k.transpose(-2, -1) / math.sqrt(d_head)
scores = scores.masked_fill(forbidden, float("-inf"))
weights = torch.softmax(scores, dim=-1)
context = weights @ v
```



Le dernier axe contient les clés.

Le masque booléen True signifie « interdit » dans ce code.

Les valeurs n'interviennent qu'après le calcul des poids.



TP 04 · VÉRIFIER L'ATTENTION



CONTEXTE

TP 04 · DONNÉES

Un exemple 2×2 calculé à la main, puis un batch multi-têtes.



EXERCICE

TRAVAIL DEMANDÉ

Implémenter et inspecter les projections, poids, masques et gradients.



BILAN

LIVRABLES

Sommes des lignes, dimensions et test de non-accès au futur.

SPE
DATA/IA

RÉSULTATS ATTENDUS



150 min

TEMPS DE L'EXERCICE



2

ÉTUDIANTS PAR BINÔME



3

PREUVES ATTENDUES



EXERCICE · UN BLOC À 256 DIMENSIONS

$d=256$, $h=8$, $d_{ff}=1024$; ignorer les biais et normalisations.

Compter les paramètres Q , K , V , O puis ceux du FFN.

Comparer leur poids relatif et donner la largeur de chaque tête.



JOUR 4 · À RETENIR

QK^T compare ; softmax normalise ; AV agrège.

Positions et masques définissent l'information accessible.

Résidus, LayerNorm et FFN complètent l'attention dans un bloc.

La justesse se vérifie par les axes, les sommes et la causalité.



05/ MODÈLES CAUSAUX ET SYNTHÈSE

Entraîner, générer, comparer et expliquer



JOUR 5 · DÉROULÉ DES 7 HEURES

Séquence	Travail attendu	Minutes
Modélisation	Objectif causal, génération et perplexité	75
Vision Transformer	Patches, position et comparaison aux CNN	60
Protocole	Ablations, limites et préparation du projet	45
TP 05	Mini-Transformer causal sur séquences synthétiques	180
Évaluation	Restitution argumentée et synthèse transversale	60

Total : 420 minutes · 7 heures de formation effective, hors pauses



OBJECTIF AUTO-RÉGRESSIF

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t})$$
$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t})$$

SPE
DATA/IA

Le modèle factorise la probabilité de la séquence.

Chaque terme ne conditionne que sur les tokens précédents.

L'entraînement minimise la log-vraisemblance négative.



DÉCALER ENTRÉES ET CIBLES

$$\text{inputs} = s_{0:T-1}, \quad \text{targets} = s_{1:T}$$



Entrée : début, $x_1, x_2, \dots, x_{t-1}$.

Cible : $x_1, x_2, \dots, x_t$.

Le masque causal ne remplace pas ce décalage.



TEACHER FORCING ET GÉNÉRATION

Au train, les préfixes contiennent les vrais tokens.

À l'inférence, le modèle réutilise ses propres sorties.

Une erreur peut donc modifier les contextes futurs.



DÉCODAGE : ARGMAX, TEMPÉRATURE ET ÉCHANTILLONNAGE

$$p_{\tau}(x_t = k \mid x_{<t}) = \text{softmax}(z/\tau)_k, \quad \tau > 0$$



Argmax produit une continuation déterministe pour un modèle fixé.

La température agit sur la concentration de la distribution.

Top-k et top-p restreignent les candidats selon des règles différentes.



PERPLEXITÉ ET NORMALISATION DE LA PERTE

$$\text{PPL} = \exp \left(-\frac{1}{M} \sum_{t \in \mathcal{V}} \log p_{\theta}(x_t \mid x_{<t}) \right), \quad M = |\mathcal{V}|$$



Avec des logarithmes naturels, la perplexité vaut $\exp(\text{entropie croisée})$.

Elle dépend des tokens, de la distribution et du protocole.

Comparer uniquement des évaluations réellement compatibles.



CACHE K/V À L'INFÉRENCE

$$M_{KV} \approx 2BLnd_{KV} \times \text{octets par element}$$



Les clés et valeurs des tokens passés peuvent être conservées par couche.

Chaque nouvelle requête consulte ce passé déjà calculé.

Le cache économise du recalcul mais occupe une mémoire croissante.



VISION TRANSFORMER : UNE IMAGE EN PATCHES

$$n = \frac{HW}{P^2}, \quad x_i \in \mathbb{R}^{P^2 C}, \quad z_i = x_i W_E + p_i$$



Découper l'image en patches non chevauchants.

Aplatir puis projeter chaque patch vers une dimension d .

Ajouter l'information de position avant les blocs encodeurs.



PATCHES : CALCULER LE COÛT

$$\left(\frac{224}{16}\right)^2 = 196, \quad \left(\frac{224}{8}\right)^2 = 784$$
$$\frac{784^2}{196^2} = 16$$



Image 224×224, patches 16×16 : 196 tokens visuels.

Avec un token de classification : 197 tokens.

Passer à des patches 8×8 donne 784 tokens visuels.



CNN ET ViT : COMPARER LES HYPOTHÈSES

Aspect	CNN standard	ViT standard
Interaction initiale	Locale, noyaux partagés	Patches puis attention globale
Position	Structure de grille dans les opérations	Encodage explicite des positions
Résolution	Champ réceptif progressif	Nombre de patches / tokens
Coût typique	Dépend des cartes et des noyaux	Attention dense quadratique en tokens
Conclusion	À valider pour la tâche	À valider pour la tâche

SPE
DATA/IA

TRANSFERT DANS UN TRANSFORMER

Adapter la tête, le régime de masque et les sorties à la tâche.

Contrôler tokenisation, longueur maximale et prétraitement.

Le fine-tuning peut être complet ou limité à certains paramètres.



LIMITES ET INTERPRÉTATION

Un poids d'attention ne constitue pas à lui seul une explication causale.

Une sortie fluide ne garantit pas l'exactitude du contenu.

Les performances dépendent de la distribution, des données et des choix d'évaluation.



TP 05 : LA TÂCHE ET SON PROTOCOLE

$\langle BOS \rangle, a, b, c, d, a, b, c, d, a, b, c, d, \langle EOS \rangle$



Apprendre des séquences où un motif de quatre symboles est répété.

Séparer les motifs complets entre train, validation et test.

Mesurer la perte globale et la précision sur les positions de copie.



TP 05 · ENTRAÎNER UN MINI-TRANSFORMER

JOUR 5 · MASTER 2



CONTEXTE

TP 05 · DONNÉES

Alphabet de huit symboles, deux couches, deux têtes, largeur 32.



EXERCICE

TRAVAIL DEMANDÉ

Vérifier le décalage, entraîner, tester la causalité puis générer une continuation.



BILAN

LIVRABLES

Perte, perplexité, précision de copie et ablation documentée.

SPE
DATA/IA

RÉSULTATS ATTENDUS



180 min

TEMPS DE L'EXERCICE



2

ÉTUDIANTS PAR BINÔME



3

PREUVES ATTENDUES



Deep Learning · Master 2 IA

116 / 125

CYBERSUP / 2026–2027

ABLATIONS : CHANGER UN FACTEUR

Modification	Hypothèse à tester	Contrôle requis
Sans position	Effet de l'information d'ordre	Même données et budget
Une seule tête	Diversité des projections	Largeur totale fixée
Plus de couches	Capacité / optimisation	Coût et normes des gradients
Sans masque causal	Détection de fuite	Test de préfixe doit échouer

SPE
DATA/IA

ÉVALUATION FINALE : EXPLIQUER ET PROUVER

Critère	Preuve	Points
Compréhension mathématique	Dérivation et dimensions	5
Implémentation	Calcul vérifié et masques corrects	5
Protocole expérimental	Séparation et sélection sans fuite	5
Analyse et limites	Erreurs, ablation et discussion	5

SPE
DATA/IA

SYNTHÈSE : UN MÊME CADRE, PLUSIEURS STRUCTURES

$$\text{donnees} \longrightarrow f_{\theta} \longrightarrow \mathcal{L} \xrightarrow{\text{backprop}} \nabla_{\theta} \mathcal{L} \xrightarrow{\text{optimiseur}} \theta'$$



MLP : mélange dense des caractéristiques.

CNN : localité et partage spatial.

Transformer : interactions dépendant du contenu.



EXERCICE DE SYNTHÈSE : AUDIT D'UN MODÈLE

Une excellente perte train accompagne une mauvaise génération.

Le modèle reçoit la même séquence en entrée et en cible.

Le masque est une matrice triangulaire de zéros et de uns ajoutée aux scores.

Identifier deux erreurs et proposer des tests qui les détectent.



LECTURES FOURNIES : FONDEMENTS ET MÉTHODE

[Jérémie Bigot — Introduction au Deep Learning](#)

[Romain Tavenard — Introduction au Deep Learning, 2025](#)

[Javiera Castillo Navarro — RCP 209, 2025–2026](#)

[Geoffrey Daniel — Réseaux de neurones et deep learning : utilisation et méthodologie](#)

[Romain Tavenard — Version HTML du cours](#)



ARTICLES : ARCHITECTURES ET OPTIMISATION

[Vaswani et al. — Attention Is All You Need, 2017](#)

[He et al. — Deep Residual Learning for Image Recognition, 2015](#)

[Dosovitskiy et al. — An Image is Worth 16x16 Words, 2020](#)

[Kingma et Ba — Adam, 2014](#)

[Loshchilov et Hutter — Decoupled Weight Decay Regularization, 2017](#)



NORMALISATION ET DOCUMENTATION DES API

[Ioffe et Szegedy — Batch Normalization, 2015](#)

[Ba et al. — Layer Normalization, 2016](#)

[Xiong et al. — On Layer Normalization in the Transformer Architecture, 2020](#)

[PyTorch 2.8 — Conv2d](#)

[PyTorch 2.8 — CrossEntropyLoss](#)

[PyTorch 2.8 — MultiheadAttention](#)



VOTRE CHECKLIST TECHNIQUE

Je peux dériver et contrôler le gradient d'un réseau à deux couches.

Je peux calculer les formes, paramètres et champ réceptif d'un CNN.

Je peux reconstruire une attention multi-têtes et son masque causal.

Je peux justifier un protocole de transfert et expliquer ses limites.



Questions, démonstrations et discussion

MERCI

